
Modül 00 — Kurulum ve Modern Java (21 → 27)

Ders Notları

Spring Boot Edu



2026-09-23

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Gerekli Araçlar	2
2.2 Kurulumu Doğrulama	2
2.3 Repo Nasıl Kullanılır?	3
2.4 Java 21'den 27'ye Neler Değişti?	3
3. Adım Adım Örnekler	4
3.1 Records	4
3.2 Sealed Types ve Kapsamlı (Exhaustive) Switch	5
3.3 Record Patterns	6
3.4 Text Blocks ve var	7
3.5 Virtual Threads	8
3.6 Scoped Values	9
3.7 Sequenced Collections	10
3.8 Stream Gatherers	11
3.9 Compact Source Files	11
3.10 Structured Concurrency (Preview)	12
4. Sık Yapılan Hatalar ve En İyi Pratikler	13
5. Özet	13
6. İleri Okuma	14

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- JDK 27, Docker ve IDE'nizi kurup kurs reposunu çalıştırmak
- Modül yapısını (lesson / exercise / solution) ve ödev akışını kullanmak
- Record, sealed type ve pattern matching ile veri odaklı, güvenli kod yazmak
- Text block, var, sequenced collections ve stream gatherers ile daha okunur kod yazmak
- Virtual thread'lerle bloklayan kodu ölçeklemek, scoped value ile bağlam taşımak
- Compact source file ile tek dosyalık Java programları çalıştırmak
- Preview özelliklerin (structured concurrency) ne olduğunu ve nasıl açıldığını bilmek

Ön koşullar: Temel Java bilgisi (sınıf, arayüz, koleksiyonlar, lambda) · **Tahmini süre:** 3 saat

2. Kavramlar

2.1 Gerekli Araçlar

Araç	Neden?	Not
JDK 27	Tüm modüller Java 27 ile derlenir	Oracle JDK, Amazon Corretto veya herhangi bir OpenJDK 27 dağıtımı
Docker	Veritabanları ve diğer servisler	Docker Desktop'a en az 8 GB RAM verin
IDE	Kod yazmak, test çalıştırmak	Java 27'yi destekleyen güncel IntelliJ IDEA veya VS Code (Java eklentileriyle)
Git	Repoyu almak	Herhangi bir sürüm

Maven kurmanız gerekmez. Repo, Maven Wrapper (./mvnw) ile gelir ve doğru Maven sürümünü kendisi indirir.

2.2 Kurulumu Doğrulama

```

java -version                                # "27" görmelisiniz
export JAVA_HOME=$(/usr/libexec/java_home -v 27) # macOS; Linux/Windows: JAVA_HOME'u
↪ JDK 27'ye ayarlayın
./mvnw -v                                    # "Java version: 27" görmelisiniz
docker info --format '{{.ServerVersion}}'      # Docker çalışıyor mu?

```

! Uyarı

java -version 27 gösterse bile Maven başka bir JDK ile çalışıyor olabilir. ./mvnw -v çıktısındaki **Java version** satırına bakın. Yanlışsa build, “This course needs JDK 27” mesajıyla durur.

2.3 Repo Nasıl Kullanılır?

Her modül modules/NN-konu/ altında aynı yapıdadır:

Klasör	İçerik
lesson/	Ders örnekleri ve testleri. Her zaman çalışır ve yeşildir.
exercise/	Ödevlerin başlangıç kodu. TODO'lar sizi bekler, testler kırmızı başlar.
solution/	Referans çözüm. Testleri exercise/ ile birebir aynıdır.
docs/	Ders notları ve ödevler, Türkçe ve İngilizce, MD ve PDF

Ödev akışı: docs/tr/odevler.md dosyasını okuyun, exercise/ içindeki TODO'ları tamamlayın, ./mvnw -Pexercises -pl modules/<modül>/exercise test ile testleri yeşile çevirin.

2.4 Java 21'den 27'ye Neler Değişti?

Java her altı ayda bir yeni sürüm çıkarır. Bir özellik önce **preview** (önizleme) olarak gelir, geri bildirimle olgunlaşır ve sonra **final** olur. Preview özellikler varsayılan olarak kapalıdır ve --enable-preview bayrağı ister.

Özellik	Durum (Java 27)	Bölüm
Records, sealed types	Final (16, 17)	3.1, 3.2

Özellik	Durum (Java 27)	Bölüm
Pattern matching for switch, record patterns	Final (21)	3.2, 3.3
Unnamed variables <code>_</code>	Final (22)	3.2, 3.3
Text blocks, <code>var</code>	Final (15, 10)	3.4
Virtual threads	Final (21); 24'ten beri synchronized pinning yok	3.5
Scoped values	Final (25)	3.6
Sequenced collections	Final (21)	3.7
Stream gatherers	Final (24)	3.8
Compact source files, instance <code>main</code>	Final (25)	3.9
Structured concurrency	Preview (7. önizleme, JEP 533)	3.10

Java 27, ayrıca G1'i tüm ortamlarda varsayılan çöp toplayıcı yapar (JEP 523) ve compact object headers'ı varsayılan olarak açar (JEP 534). Bunlar kodunuzu değiştirmeden bellek ve performans kazancı sağlar.

i Not

Kursun ders kodu yalnızca **final** özellikleri kullanır. Preview özellikler sadece bu modülde, ayrı bir Maven profilinde gösterilir (bölüm 3.10).

3. Adım Adım Örnekler

Tüm örnekleri ders sırasıyla çalıştırmak için:

```
./mvnw -pl modules/00-setup-modern-java/lesson spring-boot:run
```

Örnekler `com.springbootedu.setupmodernjava` paketinin altındadır ve kurs boyunca kullanacağımız **Kitapçı** domain'ini kullanır.

3.1 Records

Amaç: Değişmez (immutable) veri taşıyıcılarını tek satırda tanımlamak.

Bir record için alanlar, constructor, erişim metotları (`amount()`), `equals`, `hashCode` ve `toString` otomatik üretilir. **Compact constructor**, parametre listesi yazmadan doğrulama ve normalleştirme yapar:

```
public record Money(BigDecimal amount) {

    public static final Money ZERO = new Money(BigDecimal.ZERO);

    public Money {                                     // compact constructor: validate
        ↪ & normalise
        Objects.requireNonNull(amount, "amount");
        if (amount.signum() < 0) {
            throw new IllegalArgumentException("Amount must not be negative: " + amount);
        }
        amount = amount.setScale(2, RoundingMode.HALF_UP); // assigns the field after
        ↪ this block
    }

    public static Money of(String amount) {
        return new Money(new BigDecimal(amount));
    }

    public Money plus(Money other) {
        return new Money(amount.add(other.amount)); // records are immutable →
        ↪ return a new value
    }
}
```

Beklenen çıktı:

```
== 3.1 Records
Money.of("12.5") = 12.50, ISBN = 9780134685991
```

Testi: `records/MoneyTest` — değerle karşılaştırma, doğrulama ve değişmezlik.

3.2 Sealed Types ve Kapsamlı (Exhaustive) Switch

Amaç: Bir tipin tüm alt tiplerini derleyiciye bildirmek ve hiçbir durumu unutmamak.
sealed arayüz, onu kimlerin uygulayabileceğini `permits` ile listeler:

```
public sealed interface Discount permits PercentageDiscount, FixedDiscount, NoDiscount {
}
```

Derleyici tüm alt tipleri bildiği için switch **kapsamlıdır**. default dalı gerekmez. Yeni bir indirim türü eklerseniz bu metot, siz o türü ele alana kadar derlenmez:

```
public static Money apply(Discount discount, Money price) {
    return switch (discount) {
        // exhaustive: every permitted type
        ↪ is covered
        case PercentageDiscount p -> price.percentOff(p.percent());
        case FixedDiscount f -> price.minusOrZero(f.amount());
        case NoDiscount _ -> price; // "_" = unnamed variable (we don't
        ↪ need it)
    };
    // Add a fourth Discount type and this method stops compiling until you handle it.
}
```

Beklenen çıktı:

```
== 3.2 Sealed types & exhaustive switch
%10 indirim / 10% off → 180.00
30.00 TL indirim / 30.00 TRY off → 170.00
İndirim yok / No discount → 200.00
```

Testi: sealed/DiscountTest

3.3 Record Patterns

Amaç: İç içe record'ları tek adımda parçalarına ayırmak ve when ile koşul eklemek.

Record pattern, bir nesnenin bileşenlerini doğrudan değişkenlere bağlar. _ ise kullanmadığımız bileşenleri atlar. Dallar **yukarıdan aşağıya** denenir, bu yüzden özel durumlar önce gelir:

```
public static Money total(OrderLine line) {
    return switch (line) {
        case OrderLine(_, int quantity) when quantity == 0 -> Money.ZERO;
        case OrderLine(Book(_, Money price, Format format), int quantity) when format ==
        ↪ Format.EBOOK ->
            price.times(Math.min(quantity, 3)); // e-books: pay for
        ↪ at most 3 copies
        case OrderLine(Book(_, Money price, _), int quantity) when quantity >= 10 ->
            price.times(quantity).percentOff(20); // bulk order: 20% off
        case OrderLine(Book(_, Money price, _), int quantity) -> price.times(quantity);
    };
}
```

Beklenen çıktı:

```
== 3.3 Record patterns
10 × paperback = 800.00
10 × e-book    = 120.00
```

Testi: patterns/OrderLinePricingTest

3.4 Text Blocks ve var

Amaç: Çok satırlı metinleri (JSON, SQL) okunur yazmak ve yerel değişkenlerde tekrar eden tip adlarından kurtulmak.

Text block `"""` ile başlar. Kapanış tırnaklarının solundaki ortak girinti otomatik silinir:

```
public static String toJson(String title, String author, int year) {
    return """
        {
            "title": "%s",
            "author": "%s",
            "year": %d
        }"""
    ↪ quotes is removed
    .formatted(title, author, year); // indentation left of the closing
}
```

`var`, tipi sağ taraftan açıkça anlaşılan **yerel** değişkenler içindir:

```
public static String receipt(List<String> titles) {
    var lines = new StringBuilder(); // var: the type is obvious from the
    ↪ right-hand side
    for (var i = 0; i < titles.size(); i++) {
        lines.append(i + 1).append(" ").append(titles.get(i)).append('\n');
    }
    return """
        KİTAPÇI / BOOKSTORE
        %sToplam / Total: %d kitap / books
        """
    .formatted(lines, titles.size());
}
```

Beklenen çıktı:

```
== 3.4 Text blocks & var
{
    "title": "Effective Java",
    "author": "Joshua Bloch",
    "year": 2018
}
```

Testi: textblocks/CatalogExportTest

3.5 Virtual Threads

Amaç: Bloklayan (ağ, veritabanı bekleyen) kodu, reaktif programlamaya geçmeden binlerce eşzamanlı işe ölçeklemek.

Virtual thread'ler JVM tarafından yönetilen hafif thread'lerdir. Bir virtual thread beklerken (ör. Thread.sleep, soket okuma) altındaki işletim sistemi thread'i serbest kalır ve başka işler çalışır. Bu yüzden her görev için yeni bir virtual thread açmak ucuzdur:

```
public static Result lookupAll(int count, Duration latency) {
    long start = System.nanoTime();
    List<Future<Integer>> prices = new ArrayList<>();

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) { // one new
        ↪ virtual thread per task
        for (int i = 0; i < count; i++) {
            prices.add(executor.submit(() -> fetchPrice(latency))); // plain
        }
        ↪ blocking code
    }
    ↪ for all tasks
    // close() waits

    int completed = (int) prices.stream().filter(Future::isDone).count();
    return new Result(completed, Duration.ofNanos(System.nanoTime() - start));
}

private static int fetchPrice(Duration latency) throws InterruptedException {
    Thread.sleep(latency); // simulates a slow remote call; the carrier thread
    ↪ is released meanwhile
    return 42;
}
```

Beklenen çıktı (süre makineye göre değişir):

```
== 3.5 Virtual threads
10000 blocking calls of 100 ms took 135 ms
```

Sıralı çalışsaydı $10.000 \times 100 \text{ ms} \approx 17$ dakika sürerdi.

Testi: virtualthreads/PriceLookupTest

* İpucu

Spring Boot'ta `spring.threads.virtual.enabled=true` ayarı, web isteklerini, @Async

görevlerini ve zamanlanmış işleri virtual thread’lerde çalıştırır. Kursun tüm modüllerinde bu ayar açıktır (application.yaml).

3.6 Scoped Values

Amaç: “Şu anki müşteri” gibi bir bağlamı, metod parametresi olarak her katmana taşımadan paylaşmak.

ScopedValue, ThreadLocal’in değişmez ve sınırları belli alternatifidir. Değer yalnızca run(...) bloğu çalışırken geçerlidir ve blok bitince kendiliğinden kaybolur. Temizlemeyi unutmama riski yoktur:

```
public final class CurrentCustomer {

    private static final ScopedValue<String> EMAIL = ScopedValue.newInstance();

    private CurrentCustomer() {}

    public static void runAs(String email, Runnable action) {
        ScopedValue.where(EMAIL, email).run(action);    // bound only while action runs
    }

    public static String email() {
        return EMAIL.orElse("anonymous");               // unbound outside the scope
    }
}
```

Çağrı zincirinin derinlerindeki kod, değeri parametre almadan okur:

```
public void checkout(String isbn) {
    log.add(CurrentCustomer.email() + " bought " + isbn);    // no "customer" parameter
    ↪ needed
}
```

Beklenen çıktı:

```
== 3.6 Scoped values
ayse@example.com bought 978-0-13-468599-1
anonymous bought 978-1-61729-757-1
```

Testi: scopedvalues/AuditLogTest

3.7 Sequenced Collections

Amaç: Sıralı koleksiyonlarda ilk/son elemana ve ters sıraya tek tip bir API ile erişmek.

SequencedCollection, SequencedSet ve SequencedMap arayüzleri `getFirst()`, `getLast()`, `addFirst()`, `removeLast()` ve `reversed()` metotlarını getirir. “Son görüntülenen kitaplar” listesi için idealdir:

```
public class RecentlyViewed {

    private final SequencedSet<String> isbnns = new LinkedHashSet<>();
    private final int limit;

    public RecentlyViewed(int limit) {
        this.limit = limit;
    }

    public void view(String isbn) {
        isbnns.addFirst(isbn);           // already present? LinkedHashSet moves it to
        ↪ the front
        while (isbnns.size() > limit) {
            isbnns.removeLast();        // drop the oldest
        }
    }

    public List<String> newestFirst() {
        return List.copyOf(isbnns);
    }

    public List<String> oldestFirst() {
        return List.copyOf(isbnns.reversed()); // a view, no copy of the set
    }

    public Optional<String> latest() {
        return isbnns.isEmpty() ? Optional.empty() : Optional.of(isbnns.getFirst());
    }
}
```

Beklenen çıktı:

```
== 3.7 Sequenced collections
newest first [D, A, C], oldest first [C, A, D]
```

Testi: `collections/RecentlyViewedTest`

3.8 Stream Gatherers

Amaç: map/filter ile yazılamayan ara işlemleri (gruplama, kayan pencere, birikimli toplam) stream içinde yapmak.

Stream.gather(...), kendi ara işleminizi yazmanıza izin verir. JDK hazır gatherer'lar da sunar:

```
public static List<List<String>> batches(List<String> isbns, int size) {
    return isbns.stream().gather(Gatherers.windowFixed(size)).toList();    // [a,b]
    ↪ [c,d] [e]
}

public static List<Double> movingAverage(List<Integer> dailySales, int days) {
    return dailySales.stream()
        .gather(Gatherers.windowSliding(days))                            //
        ↪ [10,20,30] [20,30,40]
        .map(window ->
            ↪ window.stream().mapToInt(Integer::intValue).average().orElse(0))
        .toList();
}

public static List<Integer> runningTotal(List<Integer> sales) {
    return sales.stream().gather(Gatherers.scan(() -> 0, Integer::sum)).toList();    // 5,
    ↪ 15, 35
}
```

Beklenen çıktı:

```
== 3.8 Stream gatherers
batches [[a, b], [c, d], [e]]
moving average [20.0, 30.0]
running total [5, 15, 35]
```

Testi: gatherers/SalesStatisticsTest

3.9 Compact Source Files

Amaç: Küçük bir programı sınıf, public static ve build aracı olmadan yazıp çalıştırmak.

Bir dosya yalnızca bir main metodu içerebilir. java.base modülünün tüm paketleri otomatik import edilir ve IO.println konsola yazar:

```
void main(String[] args) {
    String name = args.length > 0 ? args[0] : "Java 27";
```

```
var books = List.of("Effective Java", "Java Puzzlers", "Modern Java in Action"); //  
    ↪ java.base is imported  
  
IO.println("Merhaba %s! / Hello %s!".formatted(name, name));  
IO.println(books.size() + " kitap / books: " + books);  
}
```

Çalıştırın:

```
cd modules/00-setup-modern-java/lesson  
java src/scripts/HelloBookstore.java Ayşe
```

Beklenen çıktı:

```
Merhaba Ayşe! / Hello Ayşe!  
3 kitap / books: [Effective Java, Java Puzzlers, Modern Java in Action]
```

Testi: compactsource/HelloBookstoreScriptTest — dosyayı gerçekten java komutuyla çalıştırır.

3.10 Structured Concurrency (Preview)

Amaç: Birbiriyle ilgili eşzamanlı işleri tek bir birim olarak yönetmek. Biri başarısız olursa diğerleri iptal edilir ve hiçbir thread “sahipsiz” kalmaz.

Bir kitabın fiyatı ve yorumları iki ayrı servisten, **aynı anda** alınır. Toplam süre, iki çağrının toplamı değil, en uzun olanıdır:

```
public BookDetails load(String isbn) throws InterruptedException, ExecutionException {  
    try (var scope = StructuredTaskScope.open()) { // default: all must  
        ↪ succeed  
        var price = scope.fork(() -> fetchPrice(isbn)); // runs in its own  
        ↪ virtual thread  
        var reviews = scope.fork(() -> fetchReviews(isbn));  
  
        scope.join(); // waits for both; if one fails, the other is cancelled and  
        ↪ ExecutionException is thrown  
  
        return new BookDetails(price.get(), reviews.get());  
    }  
}
```

Bu kod ayrı bir kaynak klasöründedir (src/preview/java) ve yalnızca preview profiliyle derlenir:

```
./mvnw -Ppreview -pl modules/00-setup-modern-java/lesson verify
```

Testi: src/preview-test/.../BookDetailsLoaderTest — iki çağrının paralel çalıştığını ve bir hatanın tüm işlemi durdurduğunu doğrular.

!! Dikkat

Preview API'ler sürümden sürüme değişebilir. Örneğin `join()` metodunun fırlattığı exception, önzilemeler arasında değişti. Preview özellikleri üretim kodunda kullanmayın.

4. Sık Yapılan Hatalar ve En İyi Pratikler

! Uyarı

Sealed bir tip üzerindeki switch'e default dalı eklemeyin. default, yeni bir alt tip eklendiğinde derleyicinin sizi uyarmasını engeller.

- **Yapın:** Değer nesnelerini (para, ISBN, adres) record yapın ve doğrulamayı compact constructor'a koyun.
- **Yapmayın:** Record'ları JPA entity'si olarak kullanmayın. Entity'ler değişebilir olmalıdır (Modül 06).
- **Yapın:** Virtual thread'leri havuza almayın, her görev için yenisini açın (`newVirtualThreadPerTaskExecutor`).
- **Yapmayın:** Aynı anda çalışan işlerin sayısını thread havuzu boyutuyla sınırlamaya çalışmayın. Gerekirse bir Semaphore kullanın.
- **Yapın:** Yeni kodda istek bağlamı için `ThreadLocal` yerine `ScopedValue` tercih edin.
- **Yapmayın:** `var`'ı tipin anlaşılmadığı yerlerde kullanmayın (`var x = service.process();`).
- **Yapın:** Tek seferlik araçlar ve denemeler için compact source file kullanın. Uygulamalar için normal Maven projesi kullanın.

5. Özet

- Kurs JDK 27, Docker ve Maven Wrapper ile çalışır. `./mvnw -v` doğru JDK'yı göstermelidir.
- Her modülde lesson/ örnekleri, exercise/ ödevleri, solution/ çözümleri içerir.
- Record, sealed type ve pattern matching birlikte güvenli ve kısa veri modelleri kurar.
- Text block, var, sequenced collections ve gatherers günlük kodu sadeleştirir.
- Virtual thread'ler bloklayan kodu ölçekler. Scoped value bağlamı güvenli taşır.
- Preview özellikler `--enable-preview` ister ve üretim için uygun değildir.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [JEP 395: Records](#) · [JEP 409: Sealed Classes](#)
- [JEP 441: Pattern Matching for switch](#) · [JEP 440: Record Patterns](#) · [JEP 456: Unnamed Variables & Patterns](#)
- [JEP 444: Virtual Threads](#) · [JEP 491: Synchronize Virtual Threads without Pinning](#)
- [JEP 506: Scoped Values](#) · [JEP 431: Sequenced Collections](#) · [JEP 485: Stream Gatherers](#)
- [JEP 512: Compact Source Files and Instance Main Methods](#)
- [JEP 533: Structured Concurrency \(Seventh Preview\)](#)
- [JDK 27 sürüm sayfası](#)